

The Text Property Graph: A Code-Inspired Representation for Security Text

1 Introduction

Security text is not only a sequence of words. A CVE description, exploit report, vendor advisory, or threat-intelligence note usually states which vulnerability is being discussed, which software is affected, which version is vulnerable, how the attack works, what impact follows, and what mitigation is recommended. These facts are not isolated keywords. They are connected through syntax, sentence order, entity references, discourse markers, and domain-specific security relations.

The Text Property Graph (TPG) is the representation used in this project to preserve those relations. A TPG is a directed, typed, attributed property graph. Its nodes represent linguistic and security objects, such as documents, paragraphs, sentences, tokens, entities, predicates, CVE identifiers, software products, versions, attack vectors, impacts, severities, and remediations. Its edges represent typed relations between those objects, including containment, dependency, sequence, coreference, predicate-argument binding, discourse relation, entity relation, and security-specific relations.

The TPG is inspired by the Code Property Graph (CPG). A CPG combines several views of source code into one graph: syntax, control flow, data flow, and dependence. The TPG transfers the same design principle to text. Instead of leaving dependency parses, named entities, coreference links, discourse relations, and security facts as disconnected outputs, it stores them as different edge types over one shared graph.

This chapter explains the problem that motivates TPG, the fundamental structure of the representation, its relationship to the CPG, and why it is useful for security-related text. The description is grounded in the project documentation and implementation, especially the files under `docs/`, `EPSS_TPG/docs/`, and `EPSS_TPG/tpg/`.

2 Problem Definition

2.1 Why Flat Text Representations Are Not Enough

Most modern NLP pipelines begin by reducing text to tokens or subword tokens. Those tokens are then embedded by a transformer and pooled or passed to a downstream model. This approach is effective for many tasks, but it does not make the structure of the text explicit. A vector representation may contain useful information, but it does not directly expose relations such as:

- which CVE affects which software product;
- which version belongs to that software;
- which attack vector leads to which impact;

- which sentence gives the mitigation;
- which pronoun or phrase refers back to a previous entity;
- which sentence explains, contrasts with, or follows another sentence.

For security text, this is a serious limitation. A vulnerability advisory is often short and dense. Its meaning depends on relations. The words `remote`, `critical`, `Apache`, and `patch` are useful, but the important question is how they are connected. A sentence saying that a remote attacker can exploit Apache is different from a sentence saying that Apache released a patch to prevent remote exploitation. A representation that only stores a token sequence has to infer this structure indirectly.

2.2 What the TPG Is Designed to Preserve

The TPG is designed to preserve the structural information that a flat token sequence hides. It keeps the following layers in one graph:

- document structure, from document to paragraph to sentence to token;
- syntactic dependency structure inside each sentence;
- sequential flow across tokens, sentences, and paragraphs;
- entity mentions and coreference chains;
- predicate-argument relations;
- discourse and rhetorical relations;
- security entities and security-specific typed relations;
- contextual embeddings when the model-based frontend is available.

The goal is not to replace transformer embeddings. The goal is to place those embeddings inside a richer structure. In the EPSS-TPG pipeline, the final graph can be exported to PyTorch Geometric as `x`, `edge_index`, `edge_type`, and `edge_attr`, so a GNN can use both node features and typed graph structure.

3 Why We Use a Property Graph

3.1 Multiple Relations on the Same Text

Text contains many relation types at the same time. A token belongs to a sentence. The same token may depend syntactically on another token, appear inside an entity span, be followed by another token, and carry a contextual embedding. A sentence may continue the previous sentence, contrast with it, or explain it. A CVE mention may be connected to a software product, a CWE, an impact, and a mitigation.

A property graph is a natural fit because it allows all of these relations to coexist. The same node can participate in several edge types without being duplicated. This is the main reason TPG is more expressive than a single sequence, a single parse tree, or a bag of extracted entities.

3.2 Typed Nodes, Typed Edges, and Properties

In a TPG, every node has a type and a property map. The implementation stores node properties such as text, lemma, part-of-speech tag, dependency relation, entity type, sentence index, token index, character offsets, domain type, confidence, source, and optional embedding. Every edge also has a type and may carry properties such as dependency label, semantic-role label, discourse label, coreference cluster, relation type, confidence, or other metadata.

This follows the same property-graph idea used by Joern’s CPG. The project implementation stores this schema in `tpg/schema/types.py`, with `NodeProperties`, `EdgeProperties`, `NodeType`, `EdgeType`, `SecurityNodeType`, `SecurityEdgeType`, and `TPGSchema`.

4 spaCy as the Linguistic Backend

4.1 Official Definition and Role

The generic TPG does not start from hand-written token rules. It starts from spaCy, which is the linguistic backend of the Level 1 graph construction. spaCy describes itself as “Industrial-strength Natural Language Processing in Python”¹. Its project description defines it as a library for advanced NLP in Python and Cython, designed for real products, with pretrained pipelines for tokenization, tagging, parsing, named entity recognition, text classification, and related NLP tasks.²

The important point for TPG is that spaCy does not only split text into tokens. Its standard processing pipeline converts raw text into a `Doc` object with linguistic annotations. The official linguistic-features documentation states that spaCy is designed so that raw text becomes a `Doc` object with annotations attached to it.³ Those annotations include token boundaries, sentence boundaries, part-of-speech tags, lemmas, dependency-parse labels, noun chunks, and named entities when the selected pipeline provides the relevant components.

4.2 How spaCy Works

At a high level, spaCy processes text through a language pipeline. The first stage is tokenization. spaCy’s tokenizer is non-destructive: the tokenized `Doc` can reconstruct the original input text. This matters for TPG because token character offsets must remain aligned with the original document.

After tokenization, trained spaCy components can add statistical annotations. For English pipelines such as `en_core_web_sm`, the relevant annotations are:

- **sentence boundaries**, exposed through `Doc.sents`;
- **tokens**, with text, lemma, part-of-speech tag, dependency relation, token index, and character offsets;
- **dependency parses**, where each token has a syntactic head and a dependency label such as `nsubj`, `dobj`, `amod`, or `prep`;
- **named entities**, exposed through `Doc.ents`, with entity labels such as organization, product, date, or geopolitical entity;

¹<https://spacy.io/>

²<https://pypi.org/project/spacy/>

³<https://spacy.io/usage/linguistic-features#tokenization>

- **noun chunks**, which spaCy defines as base noun phrases available through `Doc.noun_chunks` when a parser is present.

The dependency parser is especially important for TPG. spaCy’s dependency parse gives the syntactic backbone that becomes the DEP subgraph. In the CPG analogy, this is the closest text-side equivalent of the AST view.

4.3 How TPG Uses spaCy

The project uses spaCy through `tpg/frontends/spacy_frontend.py`. The `SpacyFrontend` attempts to load `en_core_web_sm`. If that model is unavailable, it falls back to a blank English pipeline with a sentencizer. The fallback can still split sentences, but it does not provide the same parser, POS tagger, lemmatizer, noun chunks, or named entity recognition as the trained model. Therefore, the full TPG linguistic structure depends on the trained spaCy pipeline being available.

The TPG construction uses spaCy annotations as follows:

1. The input text is split into paragraphs using blank-line boundaries.
2. Each paragraph is passed to spaCy, producing a `Doc`.
3. For every sentence in `Doc.sents`, the frontend creates a `SENTENCE` node.
4. For every spaCy token, it creates a `TOKEN` node with text, lemma, POS tag, dependency relation, entity IOB tag, entity type, sentence index, token index, and character span.
5. For every named entity in `sent.ents`, it creates an `ENTITY` node and links that entity to its tokens with `BELONGS_TO`.
6. For content verbs, it creates `PREDICATE` nodes. Auxiliary verbs are filtered out.
7. For syntactic children whose dependency labels can be mapped to semantic roles, it creates `ARGUMENT` nodes and `SRL_ARG` edges.
8. For noun chunks, it creates `NOUN_PHRASE` nodes.
9. For selected clausal dependencies such as `advcl`, `relcl`, `ccomp`, and `acl`, it creates `CLAUSE` nodes.
10. For verb phrases with auxiliaries, negation, particles, or adverbial modifiers, it creates `VERB_PHRASE` nodes.

The spaCy frontend also creates the initial graph edges. It adds `CONTAINS` edges for document hierarchy, `DEP` edges from each token’s syntactic head to its dependent, `NEXT_TOKEN` edges for token order, `NEXT_SENT` edges for sentence order, `NEXT_PARA` edges for paragraph order, and cross-sentence `NEXT_TOKEN` edges from the last token of one sentence to the first token of the next sentence.

4.4 Why spaCy Is the Right Backend for TPG

spaCy is suitable for this role because it gives the TPG a stable linguistic front end. It provides tokenization, sentence segmentation, POS tags, dependency labels, entity spans, noun chunks, lemmas, and character offsets in one consistent `Doc` object. The TPG frontend then converts those annotations into graph nodes and edges.

In other words, spaCy supplies the first layer of linguistic analysis, while TPG supplies the graph representation. spaCy answers: what are the tokens, sentences, syntactic heads, noun chunks, and entities? TPG answers: how should those objects be connected in a typed graph that can be queried, exported, and used by a GNN?

Once the spaCy frontend has produced the linguistic skeleton, a separate embedding pass attaches contextual vectors. The project uses **SecBERT**⁴, a BERT model further pre-trained on cybersecurity text, kept frozen at inference time. The embedder runs SecBERT on the raw text, aligns its WordPiece sub-tokens to spaCy token character spans, and stores a 768-dimensional contextual vector on each `TOKEN` node under `node.properties.extra["embedding"]`. Sentence-level [CLS]-derived embeddings and entity-level mean embeddings are stored on `SENTENCE` and `ENTITY` nodes by the same pass. Downstream, the GNN reads each node’s feature as the concatenation of a 13-dimensional node-type one-hot and that 768-dimensional embedding, giving a 781-dimensional node-feature vector. The detailed architecture of the GNN that consumes these features is described in the companion document `multiview_tpg_formula.tex`.

5 Fundamental Concepts of the TPG

5.1 Formal Definition

A Text Property Graph for one document can be written as:

$$G = (V, E, X, R, \tau, \pi_V, \pi_E)$$

where V is the set of nodes, E is the set of directed edges, X is the node-feature matrix, R is the edge-relation vocabulary, τ maps each edge to a relation type, and π_V, π_E are node and edge property maps.

In the PyTorch Geometric export, the graph is represented by:

- `x`: node features, including node-type one-hot features and optional contextual embeddings;
- `edge_index`: directed graph connectivity;
- `edge_type`: integer edge-type labels;
- `edge_attr`: one-hot edge-type features;
- `y`: graph-level label when the graph is used for supervised learning.

5.2 Two Practical Schema Levels

For this project, the TPG is used at two practical levels.

Level	Description
Level 1: Generic TPG	General text graph with 13 base node types and 13 base edge types. It is produced mainly by the spaCy frontend and enrichment passes.
Level 2: Security TPG	Security-aware graph that adds 12 security node types and 10 security edge types for vulnerability text.

Table 1: TPG levels used in this chapter and in the EPSS-TPG work.

⁴<https://huggingface.co/jackaduma/SecBERT>

The generic level is meant for ordinary text. The security level is the domain-specific extension used for CVE advisories and vulnerability-related text. This chapter focuses only on these two levels.

6 Connection to the Code Property Graph

6.1 What the CPG Solved for Code

The Code Property Graph was introduced for source-code analysis. A code vulnerability often cannot be detected from syntax alone. One may need to know where a function call appears, whether a guard exists before it, whether untrusted input reaches it, and which statements depend on which conditions. The CPG solves this by storing several views of the program in one property graph.

The TPG applies the same idea to text. The analogy is not that natural language is the same as source code. The analogy is that both code and text contain several structural views, and these views are more useful when they are joined into one graph.

6.2 CPG-to-TPG Mapping

The mapping from CPG concepts to TPG concepts is implemented directly in the project schema. The main mapping is shown below.

CPG view or edge	TPG analogue	Meaning in text
AST	DEP	Syntactic dependency structure
CFG (intra-block)	NEXT_TOKEN	Token order inside a sentence
CFG (cross-block)	NEXT_SENT	Sentence order within a paragraph
CFG (cross-scope)	NEXT_PARA	Paragraph order within the document
REACHING_DEF	COREF	Entity reference flow through text
ARGUMENT	SRL_ARG	Predicate-argument binding
CDG	RST_RELATION	Rhetorical relation: cause, contrast, condition
DOMINATE	DISCOURSE	General discourse continuation
CONTAINS	CONTAINS	Structural containment
BINDS_TO	BELONGS_TO	Token membership in entity or phrase
CALL	ENTITY_REL	Relation between entities through predicates
EVAL_TYPE	SIMILARITY	Semantic similarity between nodes
(no CPG analogue)	AMR_EDGE	AMR semantic relation slot

Table 2: Main CPG-to-TPG mapping used in the implementation. Source: `tpg/schema/types.py`, dictionaries `JOERN_TO_TPG_NODE_MAP` and `JOERN_TO_TPG_EDGE_MAP`.

This mapping is the core reason TPG can be described as code-inspired. It is not just a graph built from text; it is a graph whose views deliberately mirror the multi-view structure of CPGs.

7 Nodes

7.1 Generic Node Types

The generic TPG schema defines 13 node types.

Table 3: Generic TPG node types.

Node type	Role
DOCUMENT	Root node for the whole document.
PARAGRAPH	Structural container for a paragraph.
SENTENCE	Sentence-level unit.
TOKEN	Atomic linguistic token.
ENTITY	Named or extracted entity.
PREDICATE	Verb or action node.
ARGUMENT	Role-bearing argument linked to a predicate.
CONCEPT	Abstract concept or category.
NOUN_PHRASE	Noun phrase chunk.
VERB_PHRASE	Verb phrase chunk.
CLAUSE	Clause or subordinate structure.
MENTION	Coreference mention such as a pronoun.
TOPIC	Document-level topic node.

7.2 Security Node Types

The security extension adds 12 domain-specific node types.

Table 4: Security-specific TPG node types.

Node type	Role
CVE_ID	Vulnerability identifier.
CWE_ID	Weakness class identifier.
SOFTWARE	Affected software product.
VERSION	Software version.
COMPONENT	Affected component or module.
CODE_ELEMENT	Function, API, variable, or code construct mentioned in text.
ATTACK_VECTOR	Attack method or access path.
IMPACT	Consequence of exploitation.
REMEDIATION	Patch, upgrade, workaround, or mitigation.
THREAT_ACTOR	Attacker or actor type.
VULN_TYPE	Vulnerability category such as buffer overflow, XSS, or SQL injection.
SEVERITY	CVSS score or qualitative severity term.

8 Edges

8.1 Generic Edge Types

The edge vocabulary is the most important part of the TPG. It is what turns a set of text objects into a multi-view graph.

Table 5: Generic TPG edge types.

Edge type	Meaning
DEP	Syntactic dependency from head token to dependent token.
NEXT_TOKEN	Sequential token order.
NEXT_SENT	Sequential sentence order.
NEXT_PARA	Sequential paragraph order.
COREF	Coreference link between repeated mentions or pronouns and entities.
SRL_ARG	Predicate-argument relation.
AMR_EDGE	Abstract meaning relation slot in the schema.
RST_RELATION	Explicit rhetorical relation such as cause, contrast, temporal, condition, or summary.
DISCOURSE	General discourse continuation or topic-to-sentence relation.
CONTAINS	Structural containment from document to paragraph to sentence to node.
BELONGS_TO	Membership relation from entity, phrase, predicate, or argument to token.
ENTITY_REL	Relation between entities, usually inferred from nearby predicates.
SIMILARITY	Semantic similarity relation.

8.2 Security Edge Types

Security edges are optional first-class edge types emitted by `SecurityRelationsPass`. They are not only properties on generic `ENTITY_REL` edges.

Security edge	Source role	Target role
SEC_AFFECTS	CVE / vulnerability	software
SEC_HAS_VERSION	software	version
SEC_LOCATED_IN	CVE / vulnerability	component
SEC_CLASSIFIED_AS	CVE	CWE
SEC_EXPLOITED_BY	CVE / vulnerability	attack vector
SEC_CAUSES	attack vector	impact
SEC_MITIGATED_BY	CVE / vulnerability	remediation
SEC_USES_FUNCTION	CVE / vulnerability	code element
SEC_THREATENS	attack vector / threat actor	software
SEC_HAS_SEVERITY	CVE	severity

Table 6: Security relation types used by the Level 2 TPG.

When security edges are enabled, the edge vocabulary expands from 13 generic edge types to 23 total edge types. Security edges occupy indices 13–22 in the unified edge-type vocabulary. This matters for GNN training: the model can only learn from these relations when the dataset and model use the expanded edge-type vocabulary.

9 How the TPG Is Constructed

9.1 Generic Pipeline

The generic pipeline is implemented by `TPGPipeline`. It follows the same broad shape as Joern: a frontend builds the initial graph, enrichment passes add more edges and nodes, and exporters write the graph to external formats.

1. The spaCy frontend parses the text and creates document, paragraph, sentence, token, entity, predicate, argument, phrase, and clause nodes.
2. The frontend adds structural and syntactic edges such as `CONTAINS`, `DEP`, `NEXT_TOKEN`, `NEXT_SENT`, `NEXT_PARA`, `BELONGS_TO`, and `SRL_ARG`.
3. `CoreferencePass` adds `COREF` edges.
4. `DiscoursePass` adds `RST_RELATION` or `DISCOURSE` edges.
5. `EntityRelationPass` adds `ENTITY_REL` edges between entities in the same sentence, using nearby predicates when available.
6. `TopicPass` creates `TOPIC` nodes and links them to relevant sentences.
7. The SecBERT embedder runs once over the raw text, aligns its WordPiece sub-tokens to spaCy token character spans, and attaches a 768-dimensional contextual vector to every `TOKEN` (and to `SENTENCE` and `ENTITY` nodes via mean-pooling). This is the step that turns a 13-dimensional one-hot into the 781-dimensional node feature consumed by the GNN.
8. The exporter writes the graph as GraphSON or PyTorch Geometric tensors.

9.2 Security Pipeline

The security pipeline has three practical variants. All three extend, rather than replace, the generic pipeline above: the spaCy frontend always runs first, and the security frontends overlay security-specific entities and relations on top of the generic linguistic graph.

Pipeline	Frontend	Role
<code>SecurityPipeline</code>	rule-based	Regex and dictionary extraction.
<code>ModelSecurityPipeline</code>	SecBERT-based	Embeddings and similarity-based extraction.
<code>HybridSecurityPipeline</code>	rule + model	Default security pipeline when model dependencies are available.

Table 7: Security pipeline variants in the implementation.

The rule-based frontend is deterministic and strong for structured identifiers such as CVE IDs, CWE IDs, and version strings. The model-based frontend adds contextual embeddings and can identify less rigid security language. The hybrid frontend runs rule extraction first and then adds model-derived entities that do not overlap with rule-derived spans. This gives a practical balance between precision on canonical patterns and coverage for less standard wording.

10 TPG versus Tokenization

Tokenization is necessary, but it is not a complete representation of text. A tokenizer answers the question: what units does the text split into? A TPG answers a larger question: what typed objects and typed relations are present in the text?

Aspect	Tokenization	Text Property Graph
Primary unit	token or subword	typed node and typed edge
Structure	linear order	multi-relational graph
Syntax	implicit or absent	explicit DEP edges
Coreference	not represented directly	explicit COREF edges
Document layout	usually flattened	document, paragraph, sentence hierarchy
Domain relations	not represented directly	optional security nodes and edges
Model input	sequence tensor	graph tensors plus node features

Table 8: Difference between tokenization and TPG representation.

The TPG still uses tokens. Token nodes are part of the graph, and transformer embeddings may be attached to those nodes. The difference is that token embeddings are not the whole representation. They become node features inside a larger graph that preserves structural and domain relations.

11 TPG versus Knowledge Graphs

A knowledge graph usually stores normalized facts about entities. For example, it may store that one CVE affects one product or has one severity. This is useful, but a knowledge graph often abstracts away the text that produced the fact.

A TPG keeps the linguistic evidence and the extracted domain relations in the same graph. The sentence, tokens, predicates, entity spans, dependency edges, and security relation edges are present together. This makes TPG closer to a text analysis graph than to a clean factual database.

Aspect	Knowledge graph	Text Property Graph
Main goal	Store facts about entities	Preserve text structure and extracted relations
Typical node	canonical entity	document, sentence, token, entity, predicate, security object
Text evidence	often external or discarded	retained inside the graph
Syntax and discourse	usually absent	explicit edge layers
Best use	factual lookup and reasoning	graph learning, structural retrieval, explainable text analysis

Table 9: Knowledge graph versus TPG.

The security TPG can contain knowledge-graph-like facts, but it is not only a knowledge graph. It is a text-derived property graph with linguistic, document, and security layers.

12 How TPG Helps Security Text

Security meaning is relational. A useful representation should preserve relations such as:

- CVE affects software;
- software has version;
- CVE is classified as CWE;
- attack vector causes impact;
- vulnerability is mitigated by remediation;
- CVE has severity;
- vulnerability mentions a code element.

These relations are represented directly by the security edge types. They give a graph neural network more information than a list of security keywords. The model can pass messages along typed edges such as `SEC_AFFECTS`, `SEC_EXPLOITED_BY`, and `SEC_MITIGATED_BY`. This allows the graph branch to distinguish between the presence of a term and the role that term plays in the advisory.

The EPSS-TPG model also uses a multiview GNN. In that setup, edge types are grouped into views such as syntactic, sequential, semantic, discourse, and security views. This matches the CPG-inspired design: different relations are not collapsed into one adjacency matrix, but kept as separate signals that the model can weight.

13 Worked Example

Consider the sentence:

CVE-2024-1234: A buffer overflow in libfoo 1.2.3 allows remote attackers to execute arbitrary code. The vulnerability has a CVSS v3 base score of 9.8 indicating high severity.

After the linguistic frontend, security frontend, and security-relations pass, the graph contains ordinary linguistic nodes and security nodes, including:

- a `CVE_ID` node for `CVE-2024-1234`;
- a `VULN_TYPE` node for `buffer overflow`;
- a `SOFTWARE` node for `libfoo`;
- a `VERSION` node for `1.2.3`;
- an `ATTACK_VECTOR` node for `remote`;
- an `IMPACT` node for `execute arbitrary code`;
- a `SEVERITY` node for `high severity`.

The graph can then contain security edges such as:

- SEC_AFFECTS from CVE-2024-1234 to libfoo;
- SEC_HAS_VERSION from libfoo to 1.2.3;
- SEC_EXPLOITED_BY from CVE-2024-1234 to remote;
- SEC_CAUSES from remote to execute arbitrary code;
- SEC_HAS_SEVERITY from CVE-2024-1234 to high severity.

In addition, every **TOKEN** node carries a 768-dimensional SecBERT embedding obtained by averaging the WordPiece sub-token vectors whose character spans fall inside the token. The **SENTENCE** and **ENTITY** nodes carry sentence- and entity-level embeddings derived in the same step. The final node-feature vector consumed by the GNN is therefore the concatenation of the 13-dimensional node-type one-hot and the 768-dimensional contextual embedding, for a total of 781 dimensions per node.

This example shows the difference between text extraction and graph representation. The TPG does not only detect the words; it connects them with typed relations *and* attaches contextual embeddings to those connected nodes, so a downstream GNN can reason over both the structure and the local semantics simultaneously.

14 Positioning as a State-of-the-Art-Style Technique

The TPG should be described carefully. It is not enough to say that any graph representation is automatically state of the art. The contribution is more specific. TPG combines three modern ideas in one implemented pipeline:

1. it adopts the CPG principle of multi-view representation and transfers it to text;
2. it keeps transformer embeddings as node features rather than discarding neural language representations;
3. it adds a domain-specific security layer so the graph contains both linguistic structure and security semantics.

This makes TPG stronger than plain tokenization, richer than a single dependency tree, and more evidence-preserving than a clean knowledge graph. A fair claim is that this project presents a modern, CPG-inspired, multi-view graph representation for security text. Downstream performance claims should still be tied to the actual experiments.

15 Limitations

The TPG is useful, but it should not be overstated.

- Coreference is heuristic in the current enrichment pass. It uses recency and basic agreement rather than a full neural coreference system.
- Discourse relations are marker-based. They capture explicit markers such as **however**, **because**, and **subsequently**, but they do not perform full discourse parsing.

- Security extraction depends on rules, dictionaries, thresholds, and model coverage.
- Security edges are opt-in. If the pipeline does not enable security relations and the GNN edge vocabulary is not expanded, the model cannot use first-class `SEC_*` edge types.
- TPG is a representation. It improves the information available to a model, but it does not guarantee better performance on every downstream task.

16 Conclusion

The Text Property Graph represents text as a structured, typed, attributed graph. Its central idea is borrowed from the Code Property Graph: keep several views of the input in one graph rather than treating them as separate analyses. For text, those views are dependency structure, sequence, coreference, predicate-argument structure, discourse, entity relations, and optional security-specific relations.

This makes TPG different from tokenization and different from a conventional knowledge graph. Tokenization gives a sequence of units. A knowledge graph usually gives normalized facts. A TPG keeps both the textual evidence and the relations extracted from that evidence.

For security-related text, this is especially useful because the meaning of an advisory is carried by relations: which vulnerability affects which software, through which attack vector, with which impact, and with which remediation. The implemented TPG pipeline therefore provides a defensible, CPG-inspired, state-of-the-art-style representation for security text.